

TomasekValla Filestream System

A Whitepaper on Sovereign File Distribution

files.tomasekvalla.cz

Version 2.0 . September 2026

This document describes the architecture, philosophy, and current technical implementation of the TomasekValla Filestream System (TVFS), a self hosted, open source file distribution platform built on the principle that your files should be controlled by you, served by you, and deleted by you.

No cloud. No compression. No surveillance. No corporation.

1. Introduction

There is something deeply wrong with the way we share files in 2026. We have collectively handed over our photographs, our audio recordings, our videos, our documents, the digital artifacts of our lives and work, to a small number of corporations whose business model depends on owning, analyzing, and monetizing everything that passes through their servers.

We were promised convenience. We got surveillance.

We were promised storage. We got compression, transcoding, and quality degradation, all silently applied, all justified by optimizing the experience for the platform's bandwidth budget, never for the integrity of your file.

TVFS is a rejection of that bargain. It is a file distribution system that runs on hardware you control, serves files you own, at speeds that do not apologize for file size, and deletes your data on your schedule, not the platform's retention policy. This revision reflects the system as it actually runs today, not as it was first imagined.

2. The Adversaries

2.1 Corporate Cloud Storage

Google Drive, Dropbox, OneDrive, iCloud. These platforms share a common architecture: your files live on their infrastructure, subject to their terms of service, readable by their systems, and potentially accessible to advertisers, data brokers, or government requests. You are not the customer. You are the inventory.

Beyond the privacy problem, there is a quality problem. Upload a high resolution video to Google Drive and share it, the recipient downloads your file intact. Fine. But upload it to Google Photos and the story changes. Recompress to Google's preferred codec, strip metadata, resize for storage savings. The file that comes out the other end is not the file that went in.

2.2 Social Media Platforms

Discord, Instagram, X, WhatsApp, Facebook: every one of these platforms aggressively reprocesses media. Discord recompresses images and caps video quality. Instagram crushes photos and applies its own sharpening. WhatsApp strips EXIF and reduces video bitrate by a large factor.

The damage is not always visible at a glance. A casual scroll looks fine. But if you are a photographer, an audio engineer, a videographer, or anyone who cares about the technical integrity of a file, these platforms work against you.

Beyond compression, these platforms retain your content. Deleted posts surface in caches. Files uploaded years ago remain on servers you have no access to. Your data outlives your intention.

2.3 The Compression Agenda

Compression is not neutral engineering. It is a business decision made at your expense. Bandwidth costs money. Storage costs money. If a platform can cut a file's size before storing or serving it, that is infrastructure cost the platform no longer has to pay, and quality the user no longer gets to keep.

TVFS does not touch your files. What goes in comes out. The bit pattern you upload is the bit pattern that gets served. This is not a feature. It is the minimum baseline of respect for the files you created.

3. No Compression, High Fidelity

TVFS stores every file exactly as uploaded. There is no transcoding pipeline, no format normalization, no silent optimization pass. A 4K RAW video file arrives as a 4K RAW video file. A high resolution FLAC arrives intact. A large PNG with embedded color profiles and metadata is served as those exact bytes.

Files are streamed directly from the server at whatever speed the connection supports. Live streaming of high bitrate video directly from storage is not just possible, it is the default behavior. A speed reactive progress bar and background animation reflect the actual transfer rate back to the person watching it happen, rather than a generic spinner pretending nothing is measurable.

This is what file sharing looked like before platforms decided they knew better than you what your files should be.

4. Open Source as a Trust Mechanism

TVFS is fully open source. The frontend, every HTML template, every CSS rule, every client side script, is visible to anyone who loads the page. View source is not obfuscated, minified beyond reading, or hidden behind a build artifact. What you see is what runs.

The backend is maintained on GitHub as a proper, organized repository: the core server, the upload pipeline, the sharing routes, and the operational scripts, tracked as a single history rather than scattered snapshots. Personal side experiments that happen to live on the same machine are deliberately kept out of that history. What is published is TVFS, not everything that ever touched the same disk.

There is no proprietary logic. There is nothing hidden. This is not idealism. It is the only epistemically sound position when it comes to software that handles your files. Closed source file handling is a trust assertion with no evidence. Open source file handling is a verifiable claim. The difference matters.

Open source is also the only long term guarantee of software quality. Corporate SaaS products get deprecated, acquired, pivoted, or simply shut down. Open source code runs as long as someone wants to run it. TVFS can be forked, self hosted, adapted, and extended without permission from anyone.

5. Ownership of Data

When you upload a file to TVFS, it lives on a server you control. Not on infrastructure owned by a cloud provider under a shared tenancy agreement. Not in a data center whose jurisdiction you cannot verify. On hardware you chose, in a location you decided.

Files are assigned an expiration period at upload time, selectable from twelve preset windows ranging from five minutes to fourteen days. When that period expires, the file is deleted. Not archived. Not moved to cold storage for compliance purposes. Deleted. The registry entry is removed. The bytes are gone.

This is not a limitation. It is a deliberate design choice. Files on the internet that live forever create risk for the sender, for the recipient, and for anyone who might be depicted in them. Ephemeral by default is a security posture, not a technical shortcut.

For people who want a file to remain reachable across their own devices without re uploading it, a delta sync layer tracks recently uploaded files per user in a lightweight, append only log, letting a second device catch up on what changed without pulling the entire history.

5.1 Cloudflare Protection

The TVFS server sits behind Cloudflare. Among the protections this provides, one is particularly relevant in 2026: AI crawler blocking. Large language model training pipelines aggressively crawl the public internet, consuming any content they can reach to expand training datasets. TVFS files are blocked from these crawlers at the network edge.

Your files are not training data. They are not raw material for someone else's commercial product. Cloudflare's bot management enforces this boundary so that the content you share does not get ingested into a model you never consented to contribute to.

Combined with automatic expiration, TVFS files have a clearly bounded existence: available to their intended recipients for a defined period, invisible to crawlers during that period, and then gone.

6. Technical Architecture

6.1 Authentication and Sessions

Access to TVFS is gated by a tiered login system. Passwords are never stored in plaintext; each password is hashed at the moment of entry, and only the hash is stored in server configuration, kept in environment variables rather than in code or committed files. Comparison during authentication is hash to hash, so the raw password never persists beyond the single request that produces it.

Two access tiers exist. Tier one grants standard upload and download capabilities. Tier two grants elevated permissions. Upon successful authentication, the server issues a random session token mapped, server side, to a tier, a username, and an expiry, with a configurable remember duration of one to thirty days. Sessions are swept for expiry automatically, and the login control doubles as the upload entry point so returning users are not routed through a separate gate to reach the thing they came for.

6.2 Rate Limiting

Every sensitive endpoint is protected by in memory, per IP rate limiting. The authentication endpoint allows a bounded number of attempts per rolling window. Upload initiation and chunk endpoints carry their own separate limits.

Rate limit state is maintained in a Node.js Map with automatic expiry, pruned on an interval so memory usage stays bounded without introducing an external dependency such as Redis. When a limit is exceeded, the server responds with HTTP 429 and a Retry After header stating exactly how long the client should wait.

6.3 Chunked and Resumable Upload

Large files are uploaded in chunks, with a chunk size the client can configure up to roughly 99 MB, sent across multiple parallel streams to keep throughput high even on connections where a single stream would bottleneck. Each chunk is written to a temporary directory under a content addressed filename, and a per upload manifest tracks which chunks have arrived, guarded by a promise based mutex so simultaneous chunk arrivals cannot corrupt it.

If an upload is interrupted, by a network drop, a browser crash, or anything else, it does not restart from zero. The client queries the server for the manifest state and resumes by sending only the missing chunks, picking up automatically after a disconnect using state kept in the browser. A dedicated weak connection mode drops to a single small chunk stream when the network is visibly struggling, and an explicit pause and kill control lets the person uploading stop or abandon a transfer deliberately rather than just closing the tab and hoping. Unfinished chunk sessions are swept automatically after a day, so interrupted uploads do not accumulate as orphaned data.

6.4 Batch Creator and ZIP Downloader

Multiple files can be grouped into a named batch at upload time. The server generates a batch ID, associates uploaded files with it in the registry, and on completion produces a static landing page listing every file with its size, per file copy buttons for both raw and markdown formatted links, and the batch expiration date. The page requires no authentication, making it suitable for recipients with no TVFS account of their own.

From that landing page, recipients can request a ZIP of the whole batch, streamed through an archiver pipeline and served with its own expiration aligned to the source batch, or download files individually when assembling an archive would be unnecessary overhead.

6.5 File Registry

Every uploaded file and batch is tracked in a persistent JSON registry: file IDs, original and stored names, paths, MIME types, sizes, upload timestamps, and expirations. Writes are debounced by a short delay to avoid thrashing the filesystem during rapid sequential uploads, with a synchronous fallback for moments where an immediate write actually matters, such as an abort handler.

6.6 Thumbnails and Media Readers

Images and video receive generated thumbnails through a fallback chain: AVIF first for size, WebP next, JPEG as the universal fallback, produced through ffmpeg so the preview grid stays fast without touching the source file itself. A separate ffprobe backed endpoint inspects uploaded media for the technical detail that photographers, editors, and audio engineers actually want to see before trusting a file.

Plain text and Markdown files can generate a dedicated reader page rather than forcing a raw download, complete with a font switcher offering twenty self hosted fonts spanning Latin, Japanese, and monospace sets, so long form text is genuinely pleasant to read rather than technically viewable. Audio files can spin up a lightweight player page with basic tempo analysis, styled links carry the same media metadata inline, and a two row mobile toolbar with scroll based show and hide behavior keeps the reading and playback controls out of the way until they are needed.

6.7 Styled and Protected Sharing

Two earlier sharing modes, an end to end encrypted mode built on client side encryption and a separate pairing flow requiring the owner to approve each viewer, were retired in August 2026 after both accumulated instability that made them more fragile than useful. Rather than keep patching failure modes live, they were removed cleanly and documented for a future rebuild rather than resumed mid repair.

What remains, and what has proven itself the reliable option, is TVFS Users Only: a straightforward, login gated sharing mode built on the same tiered authentication as the rest of the system. It does not promise cryptographic secrecy from the server itself, and does not claim to. It promises that a file behind it is reachable only by someone who can authenticate to your TVFS instance, which for the overwhelming majority of real sharing situations, sending something to people you already trust, is

exactly the guarantee that matters.

6.8 Logging and Privacy

Requests are logged to monthly files for operational visibility, with a four month retention window after which old logs are simply gone rather than accumulating indefinitely. A dedicated privacy policy page documents this in plain language, and uploads can be marked anonymized on a per file basis so a shared link does not have to carry an identity along with it.

7. Conclusion

The centralization of file sharing was not inevitable. It was a set of economic incentives that happened to align around a handful of large platforms at a particular moment in the internet's history. Those platforms extracted enormous value, in data, in attention, in lock in, from the files people entrusted to them.

TVFS is a stake in the ground against that arrangement. It is not a product. It is not a service with a pricing page and a sales funnel. It is software that runs on hardware you own and does exactly what you tell it to do with your files. Nothing more, nothing less.

Two years in, some parts of that original vision were cut for being more trouble than they were worth, and new parts, delta sync, readers, thumbnails, a login system that finally behaves, were built because real use surfaced real needs a whitepaper written in advance could not have predicted. That is what a living system looks like. The manifesto did not change. The implementation grew up around it.

The files you share are yours. The server that serves them is yours. The schedule on which they are deleted is yours. No corporation co owns your data as a condition of transit. No algorithm recompresses your work to save money on someone else's bandwidth bill. No AI crawler trains on your content because you happened to use a public hosting platform.

That should be the default. TVFS makes it one.